DynQuant

Signal-Driven Mixed-Precision Quantization for Large Language Models

A Complete Technical White Paper

Method, On-Disk Format, CUDA Runtime, and Measured Results

Summary. DynQuant assigns a different numeric width to every weight tensor in a language model, chosen from what the model itself reveals about that tensor while it is being fine-tuned. Two per-channel second moments harvested by a training callback — the mean square of each input activation channel and of each output-gradient channel — combine with the *actual* quantization error of the *actual* candidate encoding to give a per-module, per-width sensitivity in units of loss. A greedy multiple-choice knapsack then buys bits where they are cheapest per unit of loss avoided, subject to soft quality floors and a small set of hard structural floors that no budget may breach. Weights are encoded group-wise with a float offset and an MSE-optimal per-group clip search, packed into `int32` words whose boundaries align to the group boundaries, and executed by CUDA kernels that never materialise the full-precision tensor.

On Qwen3.5-2B-Base fine-tuned for legal holding selection (CaseHOLD, $n = 5314$), the technique retains **92 %** of the fine-tuning gain at an average of **3.25 bits** — against 69 % for the ranking score published in the original work and 73.79 % absolute for a uniform 3-bit control that it beats by $+10.29$ **pts** ($p = 1.3 \times 10^{-81}$, exact paired McNemar). Peak allocated VRAM falls **4.84**×, the packed model's accuracy is *bit-identical* to the simulated one, and 417 kernel-parity tests pass under all four `compute-sanitizer` tools with zero errors and zero hazards.

Measured on NVIDIA A100 80 GB PCIe (108 SMs, 40 MB L2), CUDA 13.0.88, PyTorch 2.13.0+cu130, Transformers 5.14.1.

Every number in this document is measured, not projected. Section 11 states what is *not* established.

Contents

1 Executive summary

1.1 What the technique does

A quantized language model is a budget-allocation problem. You have a fixed number of bits to spend across a few hundred weight tensors, and the tensors are not equally deserving: some absorb aggressive rounding with no measurable effect on the loss, others are destroyed by it. Uniform quantization ignores the difference. DynQuant measures it.

The measurement happens during fine-tuning, which is when the model is already doing the forward and backward passes that expose the relevant quantities. A callback registered on the `Trainer` accumulates, per linear module, two vectors of per-channel second moments: the mean square of each input activation channel, $\mathbb{E}[x_c^2]$, of length `in_features`; and the mean square of each output-gradient channel, $\mathbb{E}[\delta_r^2]$, of length `out_features`. Nothing else is needed. From those two vectors and the weight tensor itself, the sensitivity of the loss to quantizing module m at width b is

$$\mathcal{S}_m(b) \;=\; \sum_{r,c} \mathbb{E}[\delta_r^2]\;\mathbb{E}[x_c^2]\;\big(W_{rc} - Q_b(W)_{rc}\big)^2, \tag{1}$$

a diagonal Gauss–Newton (equivalently, empirical-Fisher) estimate of the loss increase, in loss units, computed by running the weight block through the *real* encoder at the *real* candidate width, clip search included. There is no surrogate error curve and no rank transformation. Section 4 derives it.

Allocation then becomes a multiple-choice knapsack over the widths $\{2, 3, 4, 8\}$ where the price of a move is the difference between two sensitivities — what that bit actually buys: $\mathcal{S}_m(3) - \mathcal{S}_m(4)$. Every module starts at its floor; if the floors overrun the budget the allocator downgrades by cheapest damage; then it always runs an upgrade pass to reclaim the slack that discrete width steps leave on the table. Section 5.

Encoding is group-wise asymmetric affine, group 128, with an unconstrained *float* offset rather than an integer zero-point, and a per-group MSE-optimal clip search whose objective is the error of the real encoder–decoder round trip rather than of an idealised one. Section 6.

Execution keeps the weights packed. `DynQuantLinear` holds `int32` buffers in VRAM and dispatches to a CUDA GEMV templated on `<BITS, GROUP_SIZE>` for $M \le 8$ rows, falling back to tile dequant plus `F.linear` above that. Section 8.

1.2 What changed from the published formulation

The original work scored modules with an ordinal product of percentile ranks,

$$S_i \;=\; \mathrm{Rank}\big(\log(1 + \mathrm{Var}_t\|\nabla W_i\|^2)\big) \;\times\; \mathrm{Rank}\big(\mathrm{EMA}\|X_i\|\big), \tag{2}$$

and priced a move as score × params × Δerror with Δerror read off a universal 4^{-b} curve. Three things are wrong with that in practice, and all three were confirmed by measurement on this model:

1. **The activation factor points the wrong way.** $\mathrm{EMA}\|X_i\|$ is a *saliency* signal — large activations mean the tensor is busy. Correlated against the measured loss damage of quantizing each module, it comes out at $\rho = -0.2059$ globally and is positive in only 2 of 12 roles. Large-activation modules on this architecture are the *robust* ones. Multiplying a useful signal by it is worse than dropping it.
2. **The rank product destroys magnitude.** Ordinal ranks answer "which module is busier" but a knapsack needs "by how much". The shipped rank product correlates with measured damage at $\rho = +0.0040$ — statistically indistinguishable from noise.
3. **A universal error curve is not the error.** 4^{-b} is right on average and wrong per tensor. Equation 1 substitutes the measured error of the encoding that will actually be written to disk.

Equation 2 is retained, unchanged, behind the `paper-3.15` preset, so published numbers remain reproducible bit-for-bit. Equation 1 is the default.

1.3 Headline results

Table 1: CaseHOLD exact match, Qwen3.5-2B-Base, $n = 5314$. Chance is 20 % (five-way holding selection). "Retention" is the share of the $+53.35$ pts fine-tuning gain that survives quantization.

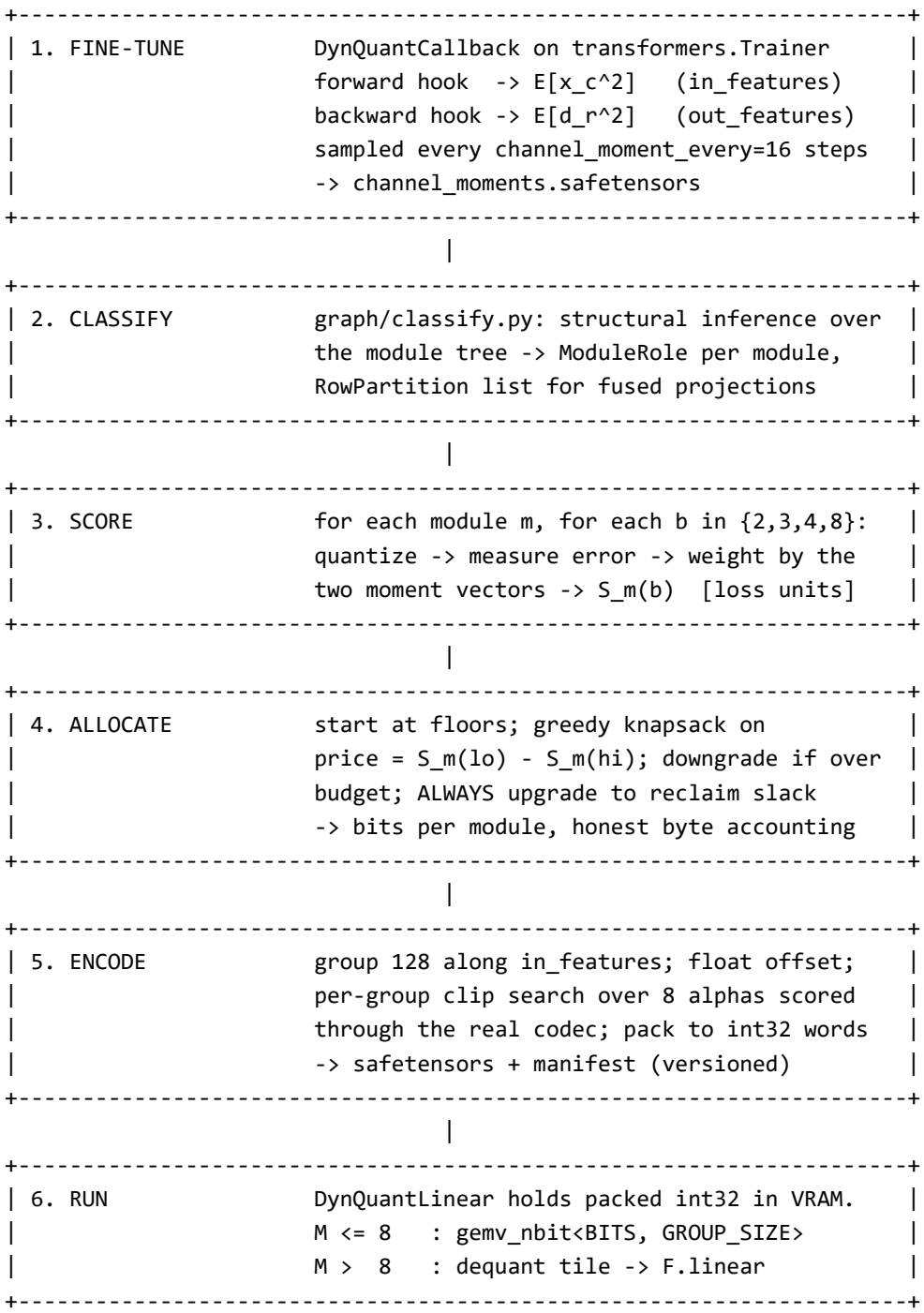
Arm	Stored bits	Size (GiB)	Exact match	Retention
Base model, fp16	16.000	3.505	35.13 %	—
Fine-tuned, fp16	16.000	3.505	**88.48 %**	100 %
DynQuant ≈4.25 bits	4.249	0.931	86.49 %	96 %
Uniform 4-bit control	4.255	0.932	86.62 %	96 %
DynQuant ≈3.25 bits, sensitivity	3.249	0.712	**84.08 %**	**92 %**
Uniform 3-bit control	3.256	0.713	73.79 %	73 %
DynQuant ≈3.25 bits, rank product (Eq. 2)	3.249	0.712	71.75 %	69 %
Guessing	—	—	20.00 %	—

Three facts carry the paper. First, at 3.25 bits the sensitivity-driven allocation beats a uniform 3-bit baseline by $+10.29$ pts and the published ranking score by $+12.33$ pts, both at $p < 10^{-80}$ on the same 5314 problems. Second, the published ranking score is not merely unhelpful at this budget — it loses to uniform by -2.03 pts ($p = 5.8 \times 10^{-5}$), which means the signal was actively misdirecting the budget. Third, at 4.25 bits neither allocation separates from uniform (-0.13 pts, $p = 0.56$): the headroom to allocate only exists once the budget is tight enough that some module must be hurt.

On the runtime side: peak allocated VRAM 0.7237 GiB against 3.5052 GiB in bf16, a **4.84**× reduction, with the allocator's predicted footprint landing within **0.03 %** of the live measurement; accuracy through the CUDA kernels *exactly* equal to the simulated path at $4468/5314$; worst observed relative error 6.96×10^{-3}; and 417 parity tests green under memcheck, initcheck, racecheck and synccheck.

And, stated plainly because it matters: decode throughput at batch 1 is **0.90**× of bf16, not faster. The kernel reaches 36–98 % of achievable HBM bandwidth depending on width, missing the ≥70 % gate at 2, 3 and 4 bits. Section 11 explains why, and why the reason is arithmetic rather than a defect.

2 The pipeline



Stages 1 and 6 touch the user's training and inference loops; stages 2–5 are offline and take minutes. Nothing in stages 2–5 needs a GPU except for speed.

3 Signals collected during fine-tuning

3.1 What is collected, and why those quantities

For a linear layer $y = Wx$ with $W \in \mathbb{R}^{\text{out}\times\text{in}}$, the gradient of the loss with respect to the weight is the outer product $\nabla_W \mathcal{L} = \delta x^\top$ where $\delta = \partial\mathcal{L}/\partial y$. That identity is the whole reason the two moment vectors suffice. A perturbation ΔW changes the loss, to second order, by

$$\Delta\mathcal{L} \;\approx\; \tfrac{1}{2}\,\mathrm{vec}(\Delta W)^\top H\,\mathrm{vec}(\Delta W), \tag{3}$$

and the Gauss–Newton approximation to H for this layer factorises as $H \approx \mathbb{E}[\delta\delta^\top] \otimes \mathbb{E}[xx^\top]$. Keeping only the diagonal of each factor — which is what makes the estimate affordable — collapses the quadratic form to exactly Equation 1: an outer product of the two second-moment vectors, contracted against the squared quantization error.

So the callback accumulates:

Table 2: Signals accumulated per module by `SignalTracker`.

Quantity	Length	Accumulation
$\mathbb{E}[x_c^2]$	`in_features`	Forward hook. Running mean of the squared input over all tokens seen on sampled steps. Stored as `input_sq`.
$\mathbb{E}[\delta_r^2]$	`out_features`	Full backward hook on the output gradient. Running mean of the square. Stored as `output_grad_sq`.
`observations`	scalar	Token count behind each mean, carried in safetensors metadata so partial runs merge correctly.
activation RMS EMA	scalar	Legacy saliency signal, $\beta = 0.99$. Retained for `paper-3.15` and for diagnostics.
gradient variance	scalar	Legacy plasticity signal, Welford online variance of $\|\nabla W\|^2$ across optimizer steps.

3.2 Cost, and the sampling gate

The two reductions are cheap in FLOPs but they are extra kernel launches on tensors that are already in cache, and there are hundreds of modules. Measured, collecting them on *every* step adds **2.30 %** of step time — which fits inside the 3 % overhead budget, but leaves nothing for anything else. Since the moments are statistical quantities whose estimates change slowly, they are sampled: the `channel_moment_every` option defaults to 16, so the hooks fire on one step in sixteen and the overhead falls by roughly that factor. The legacy scalar signals are cheap enough to collect unconditionally.

All accumulators are device-resident tensors indexed by module id; there is no `.cpu().item()` per module per step, which in the original research code caused roughly one GPU synchronisation per module per step.

3.3 Where the moments come from matters

The moments are collected on data the model has *not* memorised. Three splits were available for CaseHOLD and only one is correct:

- `test` — leaks the evaluation set into the allocation decision. Rejected.
- `train` — after fine-tuning, training loss is 0.0000 to four decimals, so $\mathbb{E}[\delta^2]$ collapses to $\sim 10^{-9}$ and the gradient factor carries no information. Rejected on measurement, not on principle.
- `validation` — used. 512 rows, 204 693 tokens. Split disjointness was verified: $|\text{validation} \cap \text{test}| = 1$ out of 5314, a duplicate present in the source dataset.

That the `train` split fails is worth dwelling on. A converged model has no gradient signal on its training data, so any Fisher-style importance measure harvested there degenerates. The signal must come from data the model still finds slightly surprising.

4 Cardinal sensitivity

4.1 The estimator

`score/sensitivity.py` computes, for each module and each candidate width, the scalar of Equation 1. The implementation never forms the [out, in] weighted product, which for the embedding table would be a 2 GB intermediate. Instead it exploits that the weighting factorises:

$$\mathcal{S}_m(b) \;=\; \underbrace{\left[E^{\circ 2}\, x^2\right]}_{\text{length out}} \cdot \delta^2, \qquad E = W - Q_b(W), \tag{4}$$

so a matrix–vector product against $\mathbb{E}[x^2]$ reduces the squared error to one number per output row, and a dot product against $\mathbb{E}[\delta^2]$ finishes it. Rows are independent because groups run along the input axis, so the computation is chunked by rows under a 64 MiB budget:

```
_ROW_CHUNK_BYTES = 64 << 20   # rows are independent (groups run along input axis)

def _module_sensitivity(weight, x2, d2, *, bits, group_size, symmetric) -> float:
    rows, cols = weight.shape
    per_row = max(cols * 4, 1)
    chunk = max(1, min(rows, _ROW_CHUNK_BYTES // per_row))
    total = 0.0
    for start in range(0, rows, chunk):
        stop  = min(start + chunk, rows)
        block = weight[start:stop]
        quantized, _ = quantize_with_search(block, bits=bits, group_size=group_size,
                                            symmetric=symmetric, row_offset=start)
        error = block.float() - quantized.dequantize(dtype=torch.float32)
        error.mul_(error)
        per_output = error @ x2f          # [chunk, in] @ [in] -> [chunk]
        total += float(per_output @ d2f[start:stop])
    return total
```

The `row_offset` argument matters: it makes the clip search deterministic and identical to what the encoder will do on the full tensor, so the sensitivity is measured on the bytes that will actually be written.

4.2 What the estimate is, and what it is not

`SensitivityTable` carries an `absolute: bool` flag, and it is `False` today. The reason is honesty about calibration. The estimate is first-order in the module: it prices the damage from quantizing module m *alone*, with everything else at full precision. Summed over a whole allocation it overshoots the measured loss increase by roughly 2×, because errors in different modules partially cancel and because the quadratic model degrades at 2-bit-sized perturbations. So the table is used as *a calibrated ordering with meaningful proportions* — module A being three times more sensitive than module B is a claim the numbers support — but not as a loss prediction. Nothing in the allocator needs the absolute scale; the knapsack only compares prices.

Two implementation details that are easy to get wrong and are handled explicitly:

- **Missing moments must not read as zero.** A module with no usable moment vectors (never routed, hook missed, shape disagreement) lands in `unestimable` and falls back to a proxy score. Treating it as zero sensitivity would make it the first thing the allocator destroys.
- **Tied embeddings need aliasing.** On a tied model, `embed_tokens.weight` and `lm_head.weight` are the same storage, but in the stored orientation only the head produces a moment pairing. `_tie_aliases` resolves the pair. Separately, `_shapes_agree` rejects wrong-axis broadcasts outright rather than letting NumPy-style broadcasting silently produce a plausible-looking wrong number.

4.3 The calibration-free control

`weight_only_sensitivity` exists as the honest control: the same formula with both moment vectors replaced by ones, i.e. pure weight reconstruction error. It requires no fine-tuning data at all. It correlates with measured damage at $\rho = +0.0645$ globally, and in the policy shootout of Section 10.5 it recovers $-29.46\,\%$ of the recoverable damage at 3.125 bits — that is, it is *worse than doing nothing*. This is the cleanest available evidence that the activation and gradient statistics are load-bearing rather than decorative.

5 Allocation

5.1 Move pricing

The allocator's central object is the price of moving one module from width `lower` to width `upper`:

```
def move_value(self, lower: int, upper: int) -> float:
    if self.sens is not None:
        lo, hi = self.sens.get(lower), self.sens.get(upper)
        if lo is not None and hi is not None:
            return max(lo - hi, 0.0)
    proxy = self.score * self.num_params * (_error_scale(lower) - _error_scale(upper))
    return max(self.fallback_scale * proxy, 0.0)
```

When a sensitivity table is present, the price *is* the difference of two measured sensitivities. Note what disappears: the percentile rank, the 4^{-b} error curve (`_error_scale`, retained only for the proxy path), and the explicit `num_params` factor — parameter count is already inside the sum of Equation 1. The proxy branch survives only for modules in `unestimable`.

5.2 The search

`allocate_bits(graph, scores, budget, policy, sensitivity)` runs three passes:

1. **Initialise at floors.** Every module starts at its role's floor width.
2. **Downgrade while over budget.** Repeatedly take the cheapest available downward move by `move_value`. If minimum-everywhere plus the hard structural floors still will not fit, raise `InfeasibleTargetError` rather than silently returning something that misses the target.
3. **Always upgrade.** Even when the floor map fits, run an upgrade pass buying the most valuable available moves until the budget is exhausted. Discrete width steps leave slack — typically a few percent of the budget — and reclaiming it is free accuracy.

Step 3 is not an optimisation, it is a correctness fix. In the original allocator, if budget$-$base$-$floors came out negative the function returned the floor map and the greedy loop never executed — so at the published 3-bit setting the score had *no effect at all* and the configuration was effectively hand-written. Soft floors plus a mandatory upgrade pass mean the score drives allocation at every target.

5.3 Floors: soft quality, hard structure

Two kinds of floor, and the distinction is the difference between a preference and an invariant.

Soft quality floors express "this role usually wants at least b bits". They can be breached if the budget demands it, and every breach is reported by role.

Hard structural floors express "below this width the architecture stops functioning". No budget breaks them:

Table 3: `STRUCTURAL_ROLES`: hard floors, unbreakable by any budget.

Role	Floor	Why it is structural
`MOE_ROUTER`	8	O(hidden × experts), often $< 0.01\,\%$ of the model. A wrong argmax routes a token through the wrong 500M parameters.
`MLA_KV_A`, `MLA_Q_A`	8	The latent bottlenecks: $\sim 1\,\%$ of the block's parameters carrying all of its information.
`LIN_ATTN_A`, `LIN_ATTN_B`	8	$\sim 0.03\,\%$ of a gated linear-attention block and they feed an exponential decay term.
`SSM_X`, `SSM_DT`	8	Same argument: negligible size, feeds an exponential.

`LM_HEAD` is *deliberately excluded* from that list, and the reason is worth stating because it looks like an omission. Its 8-bit floor is a strong preference, not an invariant. On a tied model the head *is* the embedding table — 27 % of Qwen3.5-2B's parameters — and pinning 27 % of the model at 8 bits makes every average target below ∼3.9 bits arithmetically impossible. A hard floor there would not protect quality; it would make the tool refuse to run.

5.4 Fallback calibration

Modules priced by the proxy branch and modules priced by real sensitivity live in different units, and mixing them naively means one population is systematically cheaper and gets destroyed first. `_apply_fallback_scale` rescales the proxy population by a ratio of medians — but of *rung-normalised* prices, each next-step value divided by its width span. Computing the medians on raw step values double-counts the width-span factor and yields a scale of 0.332 instead of 1.000; see defect 3 in Section 12, where on this architecture that error would have put 604 M parameters at the front of the queue for the chopping block.

5.5 Default floors by role (excerpt)

Table 4: `DEFAULT_FLOOR_BITS`, abbreviated. `UNQUANTIZED_FLOOR` keeps the tensor in its original dtype.

Role	Floor	Rationale
`EMBEDDING`	4	Errors here hit every token.
`LM_HEAD`	8	Soft; see above.
`ATTN_Q/K/V/O`, `ATTN_QKV`, `ATTN_KV`	4	Q/K errors compound through the softmax.
`ATTN_Q_GATE`	4	Fused $[Q;$ output gate]; refined per row partition.
`LIN_ATTN_QKV/Z/OUT`	4	Behaves like ordinary attention.
`LIN_ATTN_A/B`	8	Hard structural.
`LIN_ATTN_CONV`, `SSM_CONV`	—	Unquantized. Shape $[\text{ch}, 1, 4]$; identical tensors get identical answers regardless of enclosing block.
`MLP_GATE`, `MLP_GATE_UP`	4	The SwiGLU gate multiplies the up branch, so its error is amplified rather than averaged.
`MLP_UP`, `MLP_DOWN`, `MLP_FC`	3	The redundant bulk.
`MOE_ROUTER`	8	Hard structural.
`MOE_EXPERT_UP/DOWN/FC`	2	Each expert sees a fraction of tokens.
`MOE_SHARED_*`	3–4	Runs for every token, so it behaves like a dense FFN.
`MLA_Q_A`, `MLA_KV_A`	8	Hard structural.
`MLA_Q_B`, `MLA_KV_B`	4	The bulk.
`SSM_IN`, `SSM_OUT`	4	The bulk.
`SSM_X`, `SSM_DT`	8	Hard structural.
`VISION_PATCH_EMBED`	8	A few percent of a VLM but carries all image information.
`OTHER`	4	Conservative default; `dynquant inspect` warns and lists every unclassified module so nothing silently drops to 2 bits.

6 The encoder

6.1 Group-wise affine with a float offset

Weights are grouped along the input axis in blocks of `group_size` (default 128, constrained to group_size mod $32 = 0$). Each group gets a scale and an offset, and values reconstruct as

$$w \approx q \cdot \text{scale} + \text{offset}, \qquad q \in \{0, \ldots, 2^b - 1\}. \tag{5}$$

The offset is an *unconstrained float*, not an integer zero-point. This is the single highest-leverage decision in the format and it costs nothing at runtime: the dequantization $q \cdot s + o$ is one FFMA either way.

The integer-zero-point convention requires the representable range to include zero, which forces the range of any group whose values do not straddle zero to be widened all the way to it. A worked case from the format spec: a group measured at $[0.50, 0.52]$ — a perfectly ordinary outcome for a post-`SiLU` activation path or a positively-biased weight block — widens to $[0.00, 0.52]$. That is a **26.0×** range inflation, discarding **4.70 bits** of resolution at a nominal 4-bit width. There is no reason to pay it.

Scales and offsets are rounded to the storage dtype *before* the integer codes are derived, so the codes are optimal for the parameters that will actually be read back — not for higher-precision parameters that then get rounded. A degenerate group (all values equal) encodes as scale $= 0$, offset $=$ constant, $q = 0$, which round-trips exactly and cannot divide by zero.

6.2 MSE-optimal clip search

Min/max grouping is optimal for the worst case and poor for the typical case, because one outlier stretches the grid for 127 well-behaved neighbours. The encoder therefore searches a clip ratio per group:

```
CLIP_CANDIDATES = (1.0, 0.98, 0.96, 0.94, 0.92, 0.90, 0.85, 0.80)
```

Two deliberate departures from the original formulation:

1. **The objective is the real codec's error.** Each candidate is pushed through `QuantTensor.from_dense` and then `.dequantize()`, so the winning α is optimal for the bytes that reach disk — including the dtype rounding of scale and offset, and the degenerate-group handling. Scoring against an idealised affine map instead would pick a different, slightly wrong winner.
2. **Selection is per group, and the ratios are returned rather than applied.** The caller receives the chosen ratios, which makes the search reproducible and lets the sensitivity estimator evaluate exactly the same encoding the quantizer will emit.

6.3 Packing

Codes pack into `uint32` words with group boundaries aligned to word boundaries. At group 128 this gives $8/12/16/32$ words per group for $2/3/4/8$ bits. The alignment is what makes an efficient kernel possible at all: a thread block loads a fixed number of words with compile-time-constant shifts and there are no cross-lane carries. The original research code flattened the whole tensor before 3-bit packing, so group boundaries fell mid-word and no efficient kernel existed for that layout.

7 On-disk format

7.1 Versioning

Four numbers are versioned independently, because they change for independent reasons:

Version	Governs
core version	The Python API surface.
`CHECKPOINT_FORMAT_VERSION`	Layout of the weight files and manifest. A reader refuses an unknown value rather than guessing.
`STATS_SCHEMA_VERSION`	The signals sidecar. v1 (research era) migrates to v2 losslessly.
`KERNEL_ABI_VERSION`	The compiled extension's contract with core. Checked at import; mismatch is a hard error with an actionable message, not a segfault later.

`QuantTensor` stores `bits`, `group_size`, `symmetric`, `row_partitions`, `layout`, `shape` and `dtype` explicitly. None of it is inferred. The research code stored no `group_size` at all and *guessed* it from `scale.numel() // out_features` with a hardcoded 128 fallback, which breaks for `ndim` $\neq 2$ (Mamba `conv1d`, batched expert weights $[E, \text{out}, \text{in}]$) and for padded `in_features`.

7.2 Kernel chunk invariant

The GEMV processes values in chunks sized so that a whole number of 32-bit words maps to a whole number of values: $\text{kValues} \cdot \text{BITS} = \text{kWords} \cdot 32$. That single rule fixes the vector load type per width:

Table 5: Chunk geometry per width. The 3-bit case is the interesting one: 6 words is not a power of two, so it loads as `uint2` × 3 rather than `uint4`.

Bits	Words/chunk	Values/chunk	Load type
2	4	64	`uint4`
3	6	64	`uint2`
4	4	32	`uint4`
8	4	16	`uint4`

Every group is word-aligned by construction (Section 6), so scales and offsets are fetched once per group into shared memory and the inner loop is pure integer unpack plus FFMA.

8 The packed runtime

8.1 Dispatch

`DynQuantLinear` holds packed `int32` buffers, a scale tensor and an offset tensor, and never materialises the full-precision weight. Forward dispatch is on row count M:

- $M \leq$ `gemv_max_rows()` $= 8$: `dynquant::gemv_nbit<BITS, GROUP_SIZE>`, one templated instantiation per width so there is no runtime branching in the inner loop.
- $M > 8$: tile dequant into a bounded workspace, then `F.linear`. Correct and throughput-competitive immediately; the fused tensor-core path is future work.

Templating on `BITS` rather than branching is not cosmetic. The unpack shift sequence and the load type both depend on the width; a runtime branch inside the inner loop would cost more than the arithmetic it guards.

8.2 Accuracy through the kernels

The critical property of a quantization runtime is that it does not change the answer relative to the simulated path. Measured on the full CaseHOLD evaluation:

Table 6: Simulated versus packed execution, ≈3.25-bit sensitivity allocation.

Path	Correct	Exact match
Simulated (dequantized weights, `F.linear`)	4468 / 5314	84.0798 %
Packed (CUDA GEMV + dequant fallback)	4468 / 5314	84.0798 %
Difference	0	0.0000 %

Not "within noise" — identical, problem by problem. The evaluation script asserts `MATCH` and fails otherwise. Worst observed relative error on any single GEMV output element across the parity suite is 6.96×10^{-3}, consistent with fp16 accumulation of a 2-bit-quantized dot product.

8.3 Memory

Table 7: VRAM, ≈3.25-bit allocation versus bf16. "Manifest" is what the allocator predicted before anything was written.

Measurement	Packed (GiB)	bf16 (GiB)	Reduction
Manifest prediction	0.7118	—	—
Walked tensors	0.7120	3.5052	4.92×
`torch.cuda.max_memory_allocated`	0.7237	3.5052	**4.84**×
`torch.cuda.max_memory_reserved`	0.7480	3.5052	4.77×
Decode peak, batch 1	0.882	3.662	4.15×
Decode peak, batch 32	5.382	8.164	1.52×

The allocator predicted 764 317 696 bytes; live measurement was 764 530 560 bytes, an overshoot of **0.03 %**. That is the number that makes `--target-size` usable: the manifest figure is the on-disk and in-VRAM figure, with scales, offsets and packing overhead all accounted for.

At batch 32 the reduction collapses to 1.52× because activations and the KV cache dominate, which is expected and not a weight-quantization concern.

8.4 Speed

Table 8: Decode throughput, tokens/s. Above $M = 8$ the kernel is not used at all — the dequant fallback runs, and it pays for the dequant without amortising it.

Batch	Packed	bf16	Ratio
1	29.9	33.1	**0.90**×
4	116.3	126.2	0.92×
8	235.6	250.9	0.94×
32	610.2	1012.6	0.60×

Decode is slower, and the reason is not the kernel. A per-step profile isolates it:

Table 9: Per-step profile, batch 1 decode.

Metric	bf16	Packed	
GPU busy time	8.801 ms	7.763 ms	−1.038 ms
of which matmul	3.519 ms	2.486 ms	−1.033 ms, **1.42× faster**
matmul share of busy	40.0 %	32.0 %	
kernel launches	2013	1980	
wall-clock per step	28.4 ms	35.7 ms	+7.3 ms
busy share of wall	31 %	22 %	

The GPU does strictly less work with packed weights, and the entire 1.038 ms reduction in busy time is accounted for by the 1.033 ms reduction in matmul time. The loss is on the host: roughly **20–45 µs of Python and launch overhead per packed module per step**, across 187 modules. At 22 % busy share the decode loop is host-bound, so the fix is CUDA Graph capture of the decode step (planned) rather than kernel work. The 3 % slowdown on full evaluation (355.1 s packed versus 344.6 s bf16) is the same story diluted by prefill-dominated batches; batch-32 prefill is 1605 ms versus 1599 ms, i.e. unchanged.

8.5 Isolated GEMV performance

Stripping the host overhead away and timing the kernel alone against a bf16 GEMV of the same shape, and against a general (non-templated) quantized kernel:

Table 10: Isolated GEMV, $M = 1$, A100. Bandwidth is against 1665 GB/s measured achievable read bandwidth (datasheet peak is 1935 GB/s). "Gain" is speedup over the general kernel.

Module	Shape	Bits	Speedup vs bf16	% of 1665 GB/s	Gain
`out_proj`	2048 × 2048	2	1.23×	10 %	1.52×
		3	1.23×	15 %	1.55×
		4	1.18×	18 %	1.78×
		8	1.09×	33 %	1.94×
`in_proj_qkv`, `gate_proj`	6144 × 2048	2	1.26×	20 %	1.32×
		3	1.24×	29 %	1.53×
		4	1.21×	37 %	1.47×
		8	1.16×	68 %	1.63×
`down_proj`	2048 × 6144	2	1.51×	14 %	1.81×
		3	1.50×	20 %	1.85×
		4	1.40×	25 %	2.09×
		8	1.29×	44 %	2.36×
`embed/lm_head`	248320 × 2048	2	**2.56**×	36 %	1.36×
		3	**2.53**×	52 %	1.96×
		4	**2.51**×	67 %	1.61×
		8	1.89×	98 %	1.44×

The kernel is faster than bf16 at every width and shape, by 1.09× to 2.56×. It also beats a general quantized kernel by 1.32× to 2.36×, which is what the `<BITS, GROUP_SIZE>` templating buys.

But the bandwidth gate — ≥70 % of achievable — is met only at 8 bits on the large tensor (98 %). At 4, 3 and 2 bits the best figures are 67 %, 52 % and 36 %. Section 11 explains why this is arithmetic, not a bug.

8.6 Verification

Table 11: Verification of the shipping binary.

Check	Result
Kernel parity vs torch oracle, all (bits, group, shape, M)	417 passed
Full test suite	1169 passed, 34 skipped, 0 failed
`compute-sanitizer --tool memcheck`	417 passed, 0 errors
`compute-sanitizer --tool initcheck`	417 passed, 0 errors
`compute-sanitizer --tool racecheck`	417 passed, **0 hazards** (0 errors, 0 warnings)
`compute-sanitizer --tool synccheck`	417 passed, 0 errors
Register usage	64 (72 at 3 bits), `STACK:0` — no spills
Determinism	Same input → bit-identical output across runs

These ran against the exact binary that produced the performance numbers. The build was re-verified from a clean rebuild of the shipping source after late edits to `gemv.cu`: the rebuild is codegen-identical — 1384 SASS instructions in the 2-bit $M = 1$ kernel with the same `FFMA` 264 / `I2F.U16` 256 / `LOP3` 244 / `SHF` 240 histogram, the same 64 registers and `STACK:0` — and re-measures the `embed`/`lm_head` row at 2.55/2.54/2.51/1.90× and 36/52/67/99 %, the same numbers inside run-to-run noise. The 417 parity tests are what actually validates the host-side grid-expression change, since an undersized grid leaves output rows untouched rather than faulting.

9 Experimental setup

Table 12: Experimental configuration.

Model	Qwen3.5-2B-Base, `Qwen3_5ForCausalLM`
Architecture	28 layers, hybrid gated-linear / full attention, tied embeddings, fused `q_proj` $= [Q;$ output gate] (`attn_output_gate: true`)
Parameters	1.8821 B total; 1.8817 B across 187 quantizable modules
Hardware	NVIDIA A100 80 GB PCIe, 108 SMs, 40 MB L2
Software	CUDA 13.0.88, PyTorch 2.13.0+cu130, Transformers 5.14.1
Task	CaseHOLD — five-way legal holding selection, exact match, $n = 5314$
Calibration	CaseHOLD `validation`, 512 rows, 204 693 tokens
Group size	128 throughout
Widths	$\{2, 3, 4, 8\}$

9.1 Choosing the task: the headroom screen

Before spending GPU-hours, four candidate datasets were screened for whether fine-tuning could move the base model at all. The rule: a dataset is only useful if the base model is well above chance (so the task is learnable) *and* well below the supervised reference (so there is headroom for quantization to then lose).

Table 13: Headroom screen. Base accuracy measured; supervised reference from the literature.

Dataset	Base	Chance	Supervised ref.	Verdict
CaseHOLD	34.3 %	20 %	∼69 %	**Pass** — used
sql-create-context	54.0 %	0 %	∼80 %	Pass
MedMCQA	47.7 %	25 %	∼45–50 %	Reject — base already at reference
GSM8K	66.1 %	0 %	65.1 %measured	Reject — base *above* reference

GSM8K is the instructive failure. The base model already scores 66.11 %; fine-tuning moves it to 65.13 %, a change of -0.99 pts at $p = 0.48$. There is no gain, so there is nothing for quantization to retain, and every quantization arm on that task measures only degradation from a ceiling. The dataset was run anyway (Section 10.7) and its results are reported, because a flat arm is itself a finding — but the design lesson is to measure base accuracy *before* committing to a fine-tune.

10 Results

10.1 Accuracy

Table 14: CaseHOLD exact match, $n = 5314$. The base model produced 7 unparseable outputs, counted as wrong.

Arm	Correct	Exact match	Stored bits
Base, fp16	1867	35.13 %	16.000
Fine-tuned, fp16	4702	**88.48 %**	16.000
DynQuant ≈4.25 b	4596	86.49 %	4.249
Uniform 4 b	4603	86.62 %	4.255
DynQuant ≈3.25 b, sensitivity	**4468**	**84.08 %**	3.249
Uniform 3 b	3921	73.79 %	3.256
DynQuant ≈3.25 b, rank product	3813	71.75 %	3.249
Guessing	—	20.00 %	—

10.2 Paired significance

All arms were evaluated on the same 5314 problems and per-problem correctness was recorded, so every comparison below is an *exact* paired McNemar test, not a two-proportion approximation. Confidence intervals are 95 % on the paired difference.

Table 15: Exact paired McNemar, same 5314 problems throughout.

Comparison	Δ (pts)	95 % CI	p
Fine-tune vs base	+53.35	[51.87, 54.83]	$< 10^{-300}$
Cost of 4-bit quantization	−1.99	[−2.68, −1.31]	1.3×10^{-8}
Allocation vs uniform @ 4 b	−0.13	[−0.51, +0.25]	0.56 *(not separated)*
Cost of 3-bit quantization (sensitivity)	−4.40	[−5.23, −3.57]	4.8×10^{-26}
Cost of 3-bit quantization (rank product)	−16.73	—	1.1×10^{-155}
Sensitivity vs uniform @ 3 b	**+10.29**	[9.20, 11.39]	1.3×10^{-81}
Rank product vs uniform @ 3 b	**−2.03**	[−3.01, −1.05]	5.8×10^{-5}
Sensitivity vs rank product @ 3 b	**+12.33**	[11.13, 13.53]	1.7×10^{-96}

Reading the table in order:

- Fine-tuning works, enormously, and that gain is the thing being protected.
- At 4.25 bits, quantization costs 2 pts and *allocation does not matter* — the −0.13 pts difference from uniform is not distinguishable from zero. When the budget is loose enough that nothing has to be hurt, there is nothing to allocate.
- At 3.25 bits, allocation is the whole game. Sensitivity-driven allocation costs 4.40 pts where uniform costs 14.69 and the published rank product costs 16.73.
- The rank product is *significantly worse than uniform* at this budget. That is a stronger statement than "it does not help". A score that loses to no score is a score pointing the wrong way.

10.3 Retention of the fine-tuning gain

Table 16: Share of the +53.35 pts fine-tuning gain surviving each arm.

Arm	Retention
fp16	100 %
≈4.25 b	96 %
≈3.25 b, sensitivity	**92 %**
≈3.25 b, rank product	69 %

This is the framing that matters for a practitioner. You spent a fine-tune to gain 53 points. At a 4.9× compression ratio you keep 92 % of it with the shipped estimator, or 69 % with the published one — and the difference is entirely in which tensors got which widths, at identical total size.

10.4 The answer key, and what correlates with it

To evaluate a sensitivity estimator you need ground truth: the actual loss damage attributable to each module. Two such keys were built, and they answer subtly different questions. Every correlation in the literature and in this paper should be labelled with which one it used, because the numbers differ.

Table 17: The two ground-truth constructions. Neither is wrong; they measure different things.

	Key A — `marginal_3to4.json`	**Key B** — `sensitivity_3bit.json`
Construction	Whole model at 3 bits; each module *restored* to 4 bits, one at a time	Whole model at fp16; each module quantized *alone* to 3 bits
Target	Signed `gain` in loss	|Δloss|, disturbance magnitude
Asks	"what does one more bit here buy, in context?"	"how much does this module dislike 3 bits, in isolation?"

Key A is the one that matches what the allocator actually does, so it is the primary key.

Table 18: Spearman ρ against **Key A** (signed 3→4 gain). "Within-role" is the mean over 12 roles, with the count of roles in which the sign is positive.

Candidate signal	Global ρ	Within-role mean (n positive / 12)
Plasticity (gradient variance)	+0.3131	+0.266 (10/12)
Fisher / Gauss–Newton (Eq. 1)	**+0.2951**	**+0.281** (11/12)
`num_params`	+0.1492	—
Weight reconstruction error only	+0.0645	—
Gram (activation second moment only)	+0.0099	−0.151 (4/12)
Shipped rank product (Eq. 2)	**+0.0040**	−0.001 (5/12)
Saliency (EMA$\|\|X\|\|$)	**−0.2059**	−0.214 (2/12)

Table 19: Within-role Spearman ρ against **Key B** (|Δloss| from isolated 3-bit quantization), mean over 12 roles.

Candidate signal	Within-role mean	Positive roles
Gauss–Newton (Eq. 1)	**+0.521**	11/12
Plasticity	+0.491	11/12
Shipped rank product	+0.231	9/12
Weight-only	−0.227	4/12
Saliency	−0.301	2/12
Eq. 1 with $\mathbb{E}[\delta^2]$ removed	**−0.338**	1/12

The two keys agree on everything that matters, which is why both are reported:

- **Saliency anti-correlates**, on both keys, in 10 of 12 roles. Busy tensors are robust tensors on this architecture. The published formula multiplies by this.
- **The gradient factor is the load-bearing one.** Removing $\mathbb{E}[\delta^2]$ from Equation 1 does not merely weaken it — it flips the sign, from $+0.521$ to -0.338, positive in 1 role out of 12. Activation statistics alone are worse than nothing.
- **The rank product is approximately noise** on the key that matches the allocator's decision ($+0.0040$ globally, -0.001 within role, positive in 5 of 12 roles — a coin flip).
- **Plasticity and Gauss–Newton are the two real signals**, and they are close. Gauss–Newton wins on within-role consistency (11/12 on both keys) and, decisively, in the end-to-end shootout below.

For scale: fp16 loss on the calibration set is 0.1049; uniform 3-bit loss is 0.1514. So 0.0466 of loss is the entire recoverable budget that any allocation policy is fighting over.

10.5 Policy shootout

Each candidate policy was run end to end — allocate, quantize, evaluate loss — at two tight budgets. The metric is the share of uniform-3-bit damage recovered, where 100 % means matching an oracle that knows the true per-module damage. Values above 100 % are possible because the oracle is itself greedy on a first-order key.

Table 20: Share of uniform-3-bit damage recovered. Higher is better; negative means worse than uniform.

Policy	@ 3.125 bits	@ 3.250 bits
Oracle (knows the true key)	84.87 %	104.15 %
`fisher_move`	77.21 %	**105.67 %**
`fisher_diff` — ships	**85.54 %**	95.76 %
Plasticity	86.81 %	81.58 %
Shipped rank product (Eq. 2)	28.49 %	85.17 %
Weight reconstruction error only	**−29.46** %	48.51 %
`num_params`	52.01 %	26.40 %
Role-aware floors, no signal at all	65.58 %	95.08 %
Random, 5 seeds	−6.45 to 45.39 %	−5.86 to 59.15 %

The shipping policy `fisher_diff` — Equation 1 with difference pricing — is the only candidate above 85 % at *both* budgets. Three other readings deserve attention:

- **The tighter budget is the discriminating one.** At 3.250 bits, five policies land in the 85–106 % band and even "role-aware floors with no signal" reaches 95.08 %. At 3.125 bits the field spreads from

−29 % to +87 %. Evaluating an allocator only at a loose budget will fail to distinguish it from a lookup table.

- **Calibration-free weight error is actively harmful** at the tight budget (−29.46 %, worse than every random seed). Reconstruction error without the two moment vectors is not a usable importance proxy.
- **Random allocation can look respectable.** One seed reached +59.15 % at 3.250 bits. Any published allocator result at a loose budget without a random baseline should be treated as unvalidated.

10.6 What the allocation actually does differently

At an identical 3.249 stored bits, here is where the sensitivity estimator puts the budget relative to the rank product:

Table 21: Average bits per role at ≈3.25 bits. Both columns are the same total size.

Role	Sensitivity	Rank product	Δ
`k_proj`	**4.00**	3.17	+0.83
`v_proj`	**4.00**	3.50	+0.50
`q_proj` (fused Q + gate)	3.33	3.00	+0.33
`lin_attn.out_proj`	3.33	3.11	+0.22
`in_proj_qkv`	3.33	3.17	+0.16
`o_proj`	3.33	3.17	+0.16
`in_proj_z`	3.17	3.11	+0.06
`embed_tokens`	3.00	3.00	0
`in_proj_a`, `in_proj_b`	8.00	8.00	0 (hard floor)
`gate_proj`	3.12	3.12	0
`down_proj`	**2.67**	2.79	−0.12
`up_proj`	**2.67**	2.83	−0.16

The trade is legible: **take bits out of the FFN bulk and put them into K and V**. K and V projections go all the way to 4 bits — their errors compound through the softmax and then through every subsequent token's attention — paid for by pushing `up_proj` and `down_proj` below 2.7 bits average, where the FFN's redundancy absorbs it.

The width histogram is the surprise:

Table 22: Parameters at each extreme width. The *better* map is the *more* aggressive one.

Width	Sensitivity	% of model	Rank product	% of model
2-bit	440.4 M	23.4 %	220.2 M	11.7 %
4-bit	427.8 M	22.7 %	207.6 M	11.0 %

The winning allocation puts *twice* as many parameters at 2 bits and twice as many at 4 bits: it is more polarised in both directions. This refutes a reading that came out of earlier experiments — that 2-bit round-to-nearest is simply unusable and should be avoided. It is unusable in the wrong places. Put 23 % of the model at 2 bits and spend the savings on K/V, and you gain 12.33 points over the map that hedged.

10.7 The GSM8K arm

Reported for completeness, and because it is a clean negative result.

Table 23: GSM8K, $n = 1319$, exact match.

Arm	Exact match	
Base, fp16	66.11 %	
Fine-tuned, fp16	65.13 %	-0.99 pts, $p = 0.48$ — no gain
≈4.25 b DynQuant	58.15 %	(uniform 4 b: 56.79 %)
≈3.25 b DynQuant	13.57 %	(uniform 3 b: 21.46 %); allocation -7.88 pts, $p = 1.1 \times 10^{-10}$

Two things to take from this. First, the fine-tune produced no gain — the base model was already at the supervised reference — so the moments were harvested from a model with nothing to say about this task, and the allocation they produced is worse than uniform. Signals from a converged, gainless fine-tune are not informative signals. Second, chain-of-thought arithmetic degrades far more steeply under 3-bit quantization than five-way classification does: 65 % → 14 % versus 88 % → 84 %. Multi-step generative reasoning has no error tolerance, because a single wrong token invalidates the rest of the chain.

The 3.25-bit *sensitivity* arm was not run on GSM8K, so the fair comparison — does the new estimator also fail on a gainless fine-tune, or does it recover? — remains open.

11 What is not established

This section exists because everything above is a measurement and measurements have scope.

11.1 Decode is slower, and the bandwidth gate is missed

At batch 1, packed decode runs at **0.90×** bf16 throughput. The kernel itself is faster (Section 8); the loss is 20–45 µs of host overhead per module per step over 187 modules, with the decode loop only 22 % GPU-busy. CUDA Graph capture is the fix and is not yet implemented.

The bandwidth gate of ≥70 % achievable is met only at 8 bits on the largest tensor. The reason is an issue-rate floor, not a memory-system problem. Analysis on sm_80, where FFMA/FADD/integer-add/`LOP3`/`SHF` all issue at 64 per SM per clock and conversions from 8- and 16-bit integers at 64 while all other conversions issue at 16:

- The instruction mix of the 2-bit kernel gives an issue-rate floor of ∼141 µs against ∼237 µs measured — roughly 60 % issue efficiency. The kernel is *instruction*-bound, and at low bit widths there are simply more unpack instructions per byte of memory traffic.
- At $K = 2048$ with 64 values per chunk, each warp executes exactly *one* main-loop iteration. There is no second iteration to overlap with, so no intra-warp memory-level parallelism is available to hide the load latency. A rows-per-warp sweep confirms the ceiling is structural: 2 rows gives 34/50/65/98 %, 4 rows gives 36/52/67/98–99 % (shipped), 8 rows gives 36/49/64/97 % and spills at $M \geq 4$.
- The fp32 magic-number dequantization trick used by Marlin and AWQ, which would eliminate the integer→float conversions entirely, is *not available* in this format. It requires $\text{offset}' = -(2^{23} + z) \cdot \text{scale}$, which at 2 bits produces ∼17 % error. The escape hatch those kernels use exists only in fp16, and this runtime accumulates in fp32.

So the honest statement is: the memory saving is real and verified (4.84×); the speed claim is not. The kernel wins in isolation and loses in the loop.

11.2 Scope of the accuracy result

One model (Qwen3.5-2B-Base), one task (CaseHOLD), two budgets (3.25 and 4.25 bits), one group size (128). The $+10.29$ pts result is a large, robustly significant effect on that configuration and nothing has yet shown it generalises. Specifically not established:

- **Larger models.** 2B is small. The redundancy structure that makes 23 % of the model tolerable at 2 bits may be quite different at 14B or 70B.
- **MoE, MLA and SSM architectures.** The roles, floors and structural invariants are implemented and unit-tested against tiny synthetic configs, but no MoE or MLA model has been quantized and evaluated end to end.
- **Generative reasoning tasks.** The GSM8K arm suggests 3-bit is simply too aggressive for chain-of-thought, independent of allocation.
- **Task specificity of the moments.** The central premise — that signals harvested during *this* fine-tune produce an allocation specialised to *this* task — is untested. At 4.25 bits the two tasks produced 177 of 187 identical widths, which is what a task-*in*dependent allocation looks like. The isolating experiment (CaseHOLD moments versus GSM8K moments on one fixed model at one fixed budget) has not been run.

11.3 Limits of the estimator itself

- **First-order in the module.** Equation 1 prices each module as if it were the only one quantized. Summed over an allocation it overshoots measured loss by ∼2×. This is why `absolute` is `False`

and why the table is used as an ordering.

- **Diagonal Gauss–Newton.** Off-diagonal curvature within a module is discarded, as is all cross-module curvature.
- **Post-hoc, not streamed.** Moments are written at the end of fine-tuning and read back. There is no online allocation that adapts during training.
- **Sampled every 16 steps.** Chosen for cost, not accuracy. The sensitivity of the result to that interval has not been swept.

11.4 Cost

Packing 187 modules runs on CPU in 447 s. Full CaseHOLD evaluation takes 355.1 s packed versus 344.6 s bf16.

12 Defects found and fixed

Four defects are documented here because each one was silent — it produced plausible numbers rather than an error — and because each says something about where this class of system fails.

12.1 Every role's least important module was free to destroy

`percentile_ranks` mapped rank to $(\text{rank} - 1)/(n - 1)$. The minimum-ranked module in every role therefore scored *exactly* 0. Since the price of a downward move is proportional to the score, a zero score means zero damage, so the allocator would take that module to the minimum width first, every time, at no cost — regardless of what the module was.

The fix is the Hazen plotting position, $(\text{rank} - 0.5)/n$, which never reaches 0 or 1.

The instructive part is how it survived: **the scorer had no test file at all**. A score-to-rank transformation is exactly the kind of pure function that is trivially testable — ties, $n = 1$, all-equal inputs, NaN — and the absence of the test was the defect that let the other defect live.

12.2 The paired test was unrecoverable after the GPU-hours were spent

The evaluation harness stored summary counts per arm: correct, total, accuracy. That is enough for a two-proportion test and *not* enough for a paired one, and the arms differ by a couple of points on 5314 problems, where pairing is worth roughly an order of magnitude in statistical power.

By the time this was noticed, the GPU-hours were spent and the per-problem outcomes were gone. Every arm had to be re-run.

The fix is structural: the results schema now *requires* an unsampled per-problem `hits` boolean array, and `eval/compare.py` computes exact McNemar from it. Every p-value in Section 10.4 exists because of that change.

12.3 Fallback calibration double-counted the width span

`_apply_fallback_scale` equalises the proxy-priced and sensitivity-priced populations by a ratio of medians. The medians were taken over *raw* next-step values — but a next-step value already contains the width span of that step. Dividing by the span (rung-normalising) before taking the median is required; without it the computed scale came out 0.332 instead of 1.000, a factor-of-three systematic discount on one population.

The consequence is not a small accuracy wobble. A three-fold underpricing means whichever population sat at the lower floor becomes the cheapest thing in the queue and gets cut first — on this architecture, 604 M parameters.

12.4 A tied embedding table arrived on the GPU twice

`nn.Module._apply` invokes its conversion function once per registered buffer and memoizes nothing. A tied vocabulary table registered as a buffer on *both* `embed_tokens` and `lm_head` is therefore converted twice and *two independent copies* land on the device. Measured directly: 8192 bytes moved for one 4096-byte tensor.

On Qwen3.5-2B the vocabulary table is 27 % of the model, so the headline memory number would have been **0.9161 GiB instead of 0.7118 GiB** — a 27 % inflation of the central claim of the runtime work, reported in perfect good faith, with the manifest and the live measurement agreeing with each other because both were wrong the same way.

The fix is structural rather than a deduplication pass: the tied module registers no buffer at all and reaches the owner's storage through a `holder` property. There is one buffer, so there is nothing to deduplicate.

12.5 The pattern

All four are silent failures that yield plausible output: a zero that means "free" instead of "unranked"; a summary statistic that discards the pairing; a calibration constant off by 3×; a memory measurement inflated by 27 % with two independent sources agreeing. None would have been caught by inspection of the results. Each was caught by asking a specific quantitative question — what *exactly* does the minimum-ranked module score, what *exactly* does the allocator predict versus what is live — and following the answer.

13 Using it

13.1 Collecting signals

```
from dynquant import DynQuantCallback

trainer = Trainer(
    model=model,
    ...,
    callbacks=[DynQuantCallback(output_dir="stats/")],
)
trainer.train()
```

`SFTTrainer` from TRL takes the same callback. For a hand-written training loop:

```
from dynquant import track_signals

with track_signals(model, out="stats/"):
    for batch in loader:
        loss = model(**batch).loss
        loss.backward()
        optimizer.step(); optimizer.zero_grad()
```

Under DDP / FSDP / DeepSpeed the accumulators are reduced across ranks before the sidecar is written.

13.2 Inspecting, allocating, quantizing

The sidecar written above drives everything downstream. `--moments` is what selects the sensitivity estimator of Section 4; without it the allocator falls back to plasticity ranks, which is the right behaviour for a run whose moments were never collected and the wrong thing to do by accident.

```
dynquant inspect  out/merged --stats stats/ --moments stats/ \
                  --target 3.25 --uniform 3 4 --save-map maps/
dynquant quantize out/merged --map maps/ -o out/q3
dynquant eval     out/merged --task casehold --out runs/bf16.json
dynquant eval     out/q3     --task casehold --out runs/q3.json \
                  --compare runs/bf16.json
dynquant bench    --model out/merged
```

Budgets may be given as `--target` (average bits), `--target-size 6.5GiB`, or `--target-ratio 0.2`. All three are accounted honestly — scales, offsets and packing overhead included — so the manifest number is the on-disk number, verified here to 0.03 %.

Three properties of that sequence are deliberate. Passing `--save-map` to `inspect` and `--map` to `quantize` means the map that was reviewed is the map that gets applied, with no second allocation in between that could differ from the one that was looked at. `eval --compare` takes the *record* written by an earlier `--out`, not another model, because the test is paired on per-problem outcomes (Section 10.4) and it refuses to run across a differing task, split, shot count, seed or limit. And `bench` measures rectangles rather than a model end to end: decode is memory-bound, so the figure that means something is the fraction of *this card's measured* achievable bandwidth that the packed GEMV reaches, and a datasheet peak is the wrong denominator.

`dynquant inspect` prints the role / parameters / sensitivity / assigned-bits table and names every module that fell through to `OTHER`, so an unrecognised architecture is loud rather than quietly conservative. It also reports within-role concordance of assigned width with score, which is the diagnostic that caught the legacy allocator ignoring its scores outright: inverting every one of them changed zero of Qwen3-14B's 282 assigned widths, and nothing the allocator printed said so.

What `quantize` writes today. The packed-checkpoint writer is not yet implemented, so `-o` produces an ordinary `transformers` directory in which every value has been encoded to its allocated width and decoded back. Those are exactly the values a packed checkpoint would hold, so accuracy measured on that directory is the quantized model's accuracy; its size is the compute dtype's size, and the command prints the packed size alongside it on every run. The configuration in which the weights really are packed in VRAM — and therefore the only one from which a memory or throughput figure is meaningful — is `dynquant eval out/merged --map maps/`, which swaps the modules onto the packed runtime in place. Every memory number in this report was measured that way.

13.3 Loading

Today a quantized model is reconstituted in one of two ways, and which one is right depends on what is being measured. For accuracy, the directory written by `quantize` loads with stock `transformers` and no DynQuant installed at all. For memory or throughput, the packed runtime is installed over a loaded model in process:

```
from transformers import AutoModelForCausalLM
from dynquant.runtime.linear import pack_model

model = AutoModelForCausalLM.from_pretrained("out/merged", ...)
report = pack_model(model, bits, group_size=128)   # weights now packed in VRAM
```

`DYNQUANT_BACKEND=cuda|triton|torch` forces a backend; all three agree numerically, and the torch path is the correctness oracle that the compiled kernels are tested against.

Not yet implemented. The intended end state is a checkpoint that is packed on disk and loaded through `transformers` directly:

```
model = AutoModelForCausalLM.from_pretrained("my-org/qwen35-2b-dynquant-3bit")
```

with `DynQuantHfQuantizer` registered into `transformers.quantizers.auto` on import and swapping Linears *during* load, so peak host memory never reaches the full-precision size. That requires the packed-checkpoint writer and the streaming load path, neither of which is written. It is stated here as design, not as behaviour: the format (Section 7) was specified to admit it, and the runtime modules it would install are the ones measured in Section 8.

13.4 Reproducing the published numbers

The superseded scorer is preserved rather than deleted, because a claim that the replacement is better is only checkable if the thing it replaced still runs. It is reachable from the Python API:

```
from dynquant.score.importance import ScoreConfig, score_modules

report = score_modules(graph, stats, ScoreConfig(combine="rank_product"))
```

`combine = "rank_product"` is Equation 2, ordinal ranks and all; it is the ordering that scores $+0.231$ in Section 10.4 against the replacement's $+0.521$.

Not yet implemented. A single `--preset paper-3.15` flag bundling all three compatibility switches — `combine = "rank_product"`, `soft_floors = False` (an over-budget floor map returned unchanged, so the greedy loop never runs, which is the defect of Section 5) and `use_coherence = True` (the EMA term present in the research code though absent from the published formula) — is specified but not written, and the golden test that would pin it does not exist. The correct implementation is the default; compatibility is opt-in and, for the moment, opt-in one flag at a time.

14 Conclusions

On the method. Replacing an ordinal rank product with a cardinal, loss-unit Gauss–Newton sensitivity — measured through the real encoder at the real candidate width — is worth $+12.33$ pts of CaseHOLD accuracy at 3.25 bits on this model, at identical file size. Pricing knapsack moves by the sensitivity *difference* rather than by a score-times-params proxy removes two hand-chosen constructs (the percentile rank and the 4^{-b} error curve) and performs better than either.

On the signals. The gradient factor $\mathbb{E}[\delta^2]$ is the load-bearing one: removing it flips the correlation with measured damage from $+0.521$ to -0.338. The activation factor used alone anti-correlates ($\rho = -0.2059$), which means the published formula's second term was subtracting information. Two per-channel second-moment vectors, sampled one step in sixteen during a fine-tune that already happens, are enough.

On allocation budgets. Allocation only matters when the budget is tight. At 4.25 bits no policy separates from uniform. At 3.25 bits the spread between policies is 40 points of accuracy and even random allocation occasionally looks good. Any allocator claim made at a loose budget, or without a random baseline, should be treated as unvalidated.

On the runtime. Packed execution is exact — bit-identical accuracy to the simulated path — and the memory saving is real and predictable: 4.84× peak allocated, with the allocator's forecast within 0.03 % of live. The kernel beats bf16 in isolation at every width (1.09×–2.56×) and beats a non-templated quantized kernel by up to 2.36×. It does not yet make decode faster end to end, because the decode loop is host-bound

at 22 % GPU occupancy, and it misses the 70 % bandwidth gate at 2, 3 and 4 bits for a structural reason: at $K = 2048$ each warp runs one main-loop iteration, so there is no intra-warp latency hiding, and the fp16 magic-number dequantization shortcut is unavailable to an fp32-accumulating kernel.

The open question. The premise that gave the method its name — that the signals are *dynamic*, task-specific, so that a fine-tune on task T yields an allocation specialised to T — is the one thing not yet demonstrated. At 4.25 bits, two very different tasks produced 177 of 187 identical widths. Either the effect appears only under tight budgets, or the allocation is largely a function of architecture and the "dynamic" framing needs revising. The isolating experiment is small, well-defined, and next.

Provenance. Every quantitative claim in this document traces to a measurement recorded in `experiments/qwen35_2b/RESULTS.md` in the DynQuant repository, produced on the hardware and software stack named on the title page. Correlation figures are labelled with which ground-truth key produced them (Section 10.4); the two keys measure different quantities and their numbers are not interchangeable. Statements about what is not established are in Section 11 and are not hedges on the results above — they are the boundary of what was tested.